# Arrays

Computer Programming 1

*<2025-10-05 Sun>*

# Outline



# Outline




- Array: definition and use
- Array and functions
- Multi-dimensional array
- Array and pointers
- Array, pointers, and functions

# Type and Array variables




- An array is a finite sequence of consecutive elements of the same type.
- The number of elements of an array (size) is decided when you write the program and fixed.
- For an array `A` of type `T` and size `dim`, the compiler allocates `dim` consecutive cells of `sizeof(T)` bytes (static allocation).
- The variable `A`, declared as a array, represents the address of the first element of the sequence.

# Logical (and physical) Layout




| Address | Index | Value |
|---|---|---|
| 0x100 | 0 | 42 |
| 0x1004 | 1 | 13 |
| 0x1008 | 2 | 99 |
| | ... | |
| 0x1028 | 10 | 7 |




## Definition and initialization

```
type id[dim];
type id[dim] = { list_of_values };
type id[] = { list_of_values };
```

## Examples

```
double a[];     // compile-time error
double a[25];   // array of 25 doubles
char d[] = {'a', 'e', 'i', 'o'}; // dimension 4
```

# Definition of Arrays: constraints




- The size of the array must be computable at compile time:
  - explicitly, through the constant expression dim.
  - implicitly, via the size of the initialization list.
- If the initialization list is empty, the value is initialized to zero of type T.

# Operations over Arrays




The only operation defined on arrays is selection with index (subscripting).

```
identifier[exp]
```

where `exp` is an expression evaluating to an integer and `identifier` is a variable declared as an Array.

Valid uses:

- Used input, assigned, passed by reference, etc.
- E.g., `cin >> a[i];`, `a[n+3] = 3 * 2;`

# Arrays and ”for” go together




```
#include <iostream>
using namespace std;

int main() {
  const int SIZE = 10;
  int ARRAY[SIZE];

  for (int i = 0; i < SIZE; i++) {
    // whatever operation you need to do on ARRAY
  }
}
```

# Example: Read and Reverse




```
#include <iostream>
using namespace std;

int main() {
  const int size = 4;
  double a[size];

  cout << "Enter " << size << " real numbers:\n";
  for (int i = 0; i < size; i++) {
    cout << i << ": ";
    cin >> a[i];
  }

  cout << "Here they are in reverse order:\n";
  for (int i = size - 1; i >= 0; i--)
    cout << "\t a[" << i << "] = " << a[i] << endl;
  return 0;
}
```

# Do and dont's




- When looping DO NOT
  - start from 1 (instead of `0`)
  - end with `size` (instead of `size - 1`)
- DO Always use a symbolic constant you can pass and use around:

```
const int NUMBER_OF_GRADES = 10;
double grades[NUMBER_OF_GRADES];
```

# Invalid operations on Arrays




- Arithmetic, assignment, and input operations are not defined on arrays.
- Comparison and output operations are defined, but give unpredictable results.
- For all these operations, it is necessary to write ad-hoc functions.
- Why? There is a nice explanation here: Why array can't be copied (look for `aioeu`)

# Examples of Invalid Operations




```
// This is ok
int a[4] = {1, 2, 3, 4};
int b[4] = {1, 2, 3, 4};

// This is not ok
// a++;        // error at compile time
// a=b;        // error at compile time
// cin >> a;   // error at compile time

cout << (a == b) << endl;   // compilation ok, but is return false
cout << (a <= b) << endl;   // compilation ok, but unpredictable result
cout << a << endl;          // compilation ok, but unpredictable result
```

# Into the wild: out of Bounds Exceptions




- The operator `[]` allows access outside the valid range `0 .. dim - 1` of an array.
- This sets the memory into a possibly inconsistent state and might have catastrophic effect in case of error (e.g., stack smashing)
- It is the responsibility of the programmer to manage the subscripting operation and ensure it never goes outside the array's range.
- Note: Compiler flags like `-fsanitize=bounds` can help identify out-of-bounds errors at run-time.

# Living with no Boundaries is dangerous (example 1)

```
#include <iostream>
using namespace std;

int main() {
  int x[5];
  int b = 18;

  // Don't try this at home
  x[-1] = -1; // overflow
  x[5] = 5;   // overflow

  cout << "C++ won't complain ... x[5] is " << x[5] << endl;
  cout << "... but things might get nasty" << endl;
  cout << b << endl;
}
```

# Living with no boundaries is dangerous (example 2)




```
int main() {
  short a[3];

  int i = 0;
  while (true) {
    a[i++] = 1;
  }
}
```




- If `p` is of type `*T` and `i` is an integer, `p+i` is the address after `i` positions of dimension `sizeof(T)`.
- Arithmetic operations like `p++` or `p+=i` implicitly multiply the integer change by `sizeof(T)`.
- The difference between two pointers of the same type, `p2-p1`, is an integer representing the number of elements between the two addresses (implicitly divided by `sizeof(T)`).
- Layman version If `a` is a pointer to an integer:
  - `(a + 1)` means: move one integer down
  - `(a + 2)` means: move two integers down




The name of an array is implicitly equivalent to a constant pointer that points to the first element of the array itself.

# Equivalences




- The address expressions `&p[i]` and `p+i` are equivalent.
- The value expressions `p[i]` and `*(p+i)` are equivalent.
- If p points to the first element of array A:
  - `p+i` = &(A[i])= and `*(p+i)` = A[i]=.

# Array Size

When A is an array, its size can be calculated with:

```
int size = sizeof(A) / sizeof(T)
int size = *(&A + 1) - A;
```

| Address | Index | Value |
|---|---|---|
| 0x1000 | 0 | 42 |
| 0x1004 | 1 | 13 |
| 0x1008 | 2 | 99 |
| | ... | |
| 0x1028 | 10 | 7 |

# Example: Array and Memory




```
#include <iostream>
using namespace std;

int main() {
  int x[5];

  cout << "Array size using sizeof" << endl;
  cout << sizeof(x) / sizeof(int) << endl;

  cout << "Array size as difference between addresses of x[1] and x[0]" << endl;
  int size = *(&x + 1) - x; ^^I^^I^^I^^I
  cout << size <<endl;

  cout << "Where is the array?" << endl;
  cout << "&x and &(x + 0) are " << &x << " x is " << x << endl;
  cout << "&(x + 1) is " << (&x+1) << " (x + 1) is " << (x + 1) << endl;
}
```

# Example 1: Arrays and Pointer Arithmetic




```
#include <iostream>
using namespace std;

int main() {
  short a[3] = { 1, 2, 3 };

  for (int i = 0; i < 3; i++) {
    cout << "a[" << i << "] lives at: " << &a[i] << endl;
  }

  cout << "We can access using the index" << endl;
  for (int i = 0; i < 3; i++) {
    cout << a[i] << endl;
  }

  cout << "We can use pointer arithmetic" << endl;
  for (int i = 0; i < 3; i++) {
    cout << (a + i) << " -> " << *(a + i) << endl;
  }
}
```

# Live with no boundaries (revisited)




- C++ does not check for boundaries (unless you use `-fsanitize=bounds`)
- Now we use pointers to understand how to nastily do a side effect

# Example




```
#include <iostream>

using namespace std;

int main() {
  short a[3];
  short b = 256;

  for (int i = 0; i < 3; i++) {
    cout << "a[" << i << "] lives at: " << &a[i] << endl;
  }

  cout << "b    lives at: " << &b << endl;

  cout << "Address of b - address of a[0] is " << &b - &a[0] << endl;
  // computer outputs -4
  cout << "Hence a[-4] is: " << a[-4] << endl;

  // if you try to access b like this, you will most likely be fired
}
```

# Examples




Various examples available in the zip uploaded with the slides.

# Functions with Arrays




A function can have a formal parameter of the type array of T

```
float sum(float v[], int n) {...}
```

Arrays are fundamentally different from scalar variables and we need to pay attention at what happens.

# Functions with Arrays

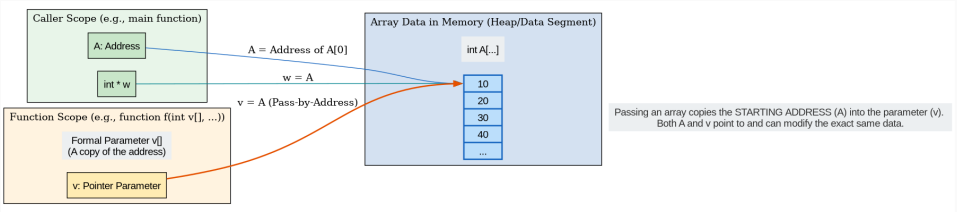



- When an array is passed to a function, only the address of the first element is copied.
- The array's elements are not copied.
- This is equivalent to passing the elements of the array by reference/pointer (pointer decay).
- This is the reason
  1. we can omit size in the square brackets: it would not be used by C++ in any case.
  2. we usually pass a second argument, the array size (or, at least, the portion of the array we are interested in)

```
float sum (float v[], int n)
```

# Functions with Arrays

(Generated by Gemini.)



# Passing Constant Arrays




- You can prevent the array from being changed by using the `const` keyword
- Syntax: `(const type identifier[], ...)`
- E.g., `int print(const int v [], ...)`
- This is efficient (no waste of CPU/memory) and ensures the array is used as read-only input

# Example (what happens with `const`)




```
#include <iostream>
using namespace std;

void try_to_modify(const int array[], int size) {
  array[0] = -10;  // c++ does not like this
}

int main() {
  int a[] = { 1, 2, 3 };
  try_to_modify(a, 3);
}
```

## Why we need to pass the size




Inside a function, the argument *decays* to a pointer (same reason we can't copy arrays… compatibility with B … the language before C)

# Example




```
#include <iostream>
using namespace std;

void pointer_decay(int array[]) {
  cout << "Functions know nothing." << endl;
  cout << "Here the size of 'array' is: " << sizeof(array) << endl;

  int * b;
  cout << "A pointer, in fact, has size: " << sizeof(b) << endl;
}

int main() {
  int a[] = { 1, 2, 3 };

  cout << "Main knows about 'a'. Its size is: " << sizeof(a) << endl;
  cout << "In fact, an integer is " << sizeof(a[0]) << " (and 4 * 3 = 12)" << endl;
  pointer_decay(a);
}
```

# Equivalent Function Signatures (Simple Array)




The following are equivalent:

```
int f(int arr[dim]);
int f(int arr[]);
int f(int *arr);
int f(const int *arr); // nearly equivalent to the others
```




**Definition**

- Arrays of arrays, generating multi-dimensional arrays (matrices).
- The size of all dimensions of an array, except the first one, must be specified.
  - Example of error: `int a[DIM1][]` ...=.
- Missing elements in the initialization list are replaced with zeros.

# Multi dimensional Arrays initialization




## Syntax

```
type id[dim1][dim2]...[dimN];
type id[dim1][dim2]...[dimN] = { list_of_values };
type id[][dim2]...[dimN] = { list_of_values };
```

## Example

```
int MAT[2][3];
int MAT[2][3] = { {1, 2, 3}, {4, 5, 6} };
int MAT[][3] = { {1, 2, 3}, {4, 5, 6} };
```

## Logical layout

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |

# Using multi-dimensional arrays




Nest ”for” loops.

```
// how you call them is arbitrary
const int ROWS = 10;
const int COLS = 20;

// empty with zeroes
int a[ROWS][COLS] = { {1, 2, 3}, {3, 4, 5} };

for (int i = 0; i < ROWS; i++) {
  for (int j = 0; j < COLS; j++) {
    cin >> a[i][j];
  }
 }
```

# Example




```
#include <iostream>
#include <iomanip>
using namespace std;

const int DIM1 = 3;
const int DIM2 = 5;

void print(const int a[][DIM]) {
  for (int i = 0; i < DIM1; i++) {
    for (int j = 0; j < DIM2; j++)
      cout << " " << setw(4)  << a[i][j];
    cout << endl;
  }
}

int main() {
  int a[DIM1][DIM2]= { {1,2,3,4,5}, {6,7,8,9,10}, {11,12,13,14,15} };
  print(a);
  return 0;
}
```

# Memory Layout




```
int MAT[2][3];
```

## Logical layout

| | | |
|---|---|---|
| MAT[0][0] | MAT[0][1] | MAT[0][2] |
| MAT[1][0] | MAT[1][1] | MAT[1][2] |

## Physical Layout

| |
|---|
| MAT[0][0] |
| MAT[0][1] |
| MAT[0][2] |
| MAT[1][0] |
| MAT[2][0] |
| MAT[3][0] |

# Physical Layout and Addresses

| | | | |
|---|---|---|---|
| MAT, MAT[0] \rightarrow | MAT[0][0] | 0x1000 | 42 |
| | MAT[0][1] | 0x1004 | 99 |
| | MAT[0][2] | 0x1008 | 109 |
| MAT[1] \rightarrow | MAT[1][0] | 0x10a2 | 12 |
| | MAT[2][0] | 0x10b0 | 14 |
| | MAT[3][0] | 0x10b4 | 123 |

# Example




```
#include <iostream>
using namespace std;

int main() {
  int MAT[3][2];

  cout << "MAT lives here " << MAT << endl;
  for (int i = 0; i < 3; i++) {
    cout << "MAT[" << i << "] lives here " << &MAT[i] << endl;
    for (int j = 0; j < 3; j++) {
      cout << "MAT[" << i << "][" << j << "] lives here " << &(MAT[i][j]) << endl;
    }
  }

  // sizes
  cout << sizeof(MAT) / sizeof(int) << endl;
  cout << sizeof(MAT[0]) / sizeof(int) << endl;
  cout << sizeof(MAT[0][1]) << endl;

}
```

# Example: sizes




```
#include <iostream>
using namespace std;

int main() {
  int MAT[3][2];

  // sizes
  cout << sizeof(MAT) / sizeof(int) << endl;
  cout << sizeof(MAT[0]) / sizeof(int) << endl;
  cout << sizeof(MAT[0][1]) << endl;

}
```

# Why we need all dimensions but the last?




To allow for pointer arithmetic

```
#include <iostream>
using namespace std;

int main() {
  int MAT[3][2];

  cout << MAT << endl;
  // &MAT + 1 moves by a full matrix
  cout << &MAT + 1 << endl;

  // &MAT[0] + 1 moves by one row
  cout << (&MAT[0] + 1) << endl;
  cout << MAT[1] << endl;
}
```

# Example in function (pointer decay)




```
#include <iostream>
using namespace std;

void f(int d[][2]) {
  // here we do not know how many rows there are
  // (this is similar to what was happening with simple arrays)
  cout << d << endl;
  cout << &d + 1 << endl;

  // however, we know how to move row by row
  // (because we have passed the dimensions)
  cout << (&d[0] + 1) << endl;
  cout << d[1] << endl;
}

int main() {
  int MAT[3][2];
  f(MAT);
}
```




Recall:

- In function calls, only the address to the first location of the array is copied.
- A formal array parameter can be specified using either the array notation or the pointer notation.

# Equivalent Function Signatures (Simple Array)




```
int f(int arr[dim]);
int f(int arr[]);
int f(int *arr);
int f(const int *arr);
```

# Equivalent Function Signatures (Multi-dimensional Array)




The dimension of all but the first array dimension must be specified.

```
int f(int arr[dim1][dim2]...[dimN]);
int f(int arr[][dim2]...[dimN]);
int f(int *arr[dim2]...[dimN]);
int f(const int *arr[dim2]...[dimN]);
```

# Returning an Array




- A C++ can return an array, in the form of a pointer
- You need to be sure that the array is still allocated when the function finishes (i.e., the array is either `static` or declared outside the function)

# Example 1: Returning a Modified Array Passed as Argument (Valid)




- If an array is passed as an argument, the function receives a pointer to the original array, and can modify it and return its address. The array is allocated externally (in the caller's scope)

# Example 1 (Valid)




```
const int dim = 10;

// v[] is the array passed to the function
int *times(int v[], int dim, int k) {
  for (int i=0; i<dim; i++)
    v[i] = v[i]*k;
  return v; // returning the address of the modified original array
}

int main()
{
  int v[dim] = {1,2,3,4,5,6,7,8,9,10};
  int * w;
  w = times(v, dim, 4);
  // v and w both point to the same array: {4, 8, 12, 16, 20, 24, 28, 32, 36, 40}
}
```

# Example 2: Incorrectly Returning a Local Array (Error)




- Returning a pointer to an array allocated locally on the stack inside the function results in undefined behavior (e.g., a segmentation fault).
- The local array is destroyed (released) when the function returns.

# Example 2 (Error)




```
int *times(int v[], int dim, int k) {
  int w[dim]; // Local array, allocated on stack

  for (int i=0; i<dim; i++) {
    w[i] = v[i] * k;
  }

  return w; // ERROR: returning a pointer to a local variable
}
```

# Example 3: Returning a Global/Static Array (Valid, but BAD idea)




It is valid to return a pointer to an array that is statically allocated internally to the function. This array is not destroyed when the function returns.

# Example 3 (Valid, but BAD idea)




```
const int dim = 10;

// Note: "a" is statically allocated externally to the function
int *times(int k) {
  static int a[dim] = {1,2,3,4,5,6,7,8,9,10}; // Global/static array

  for (int i=0; i<dim; i++)
    a[i]=a[i] * k;
  return a;
}
```

# Why it is a BAD idea




- The array is allocated once.
- Every time you call the function you are not allocating a new array: you are modifying the same data (so, calling the functions with the static array seems to be changing a variable outside the functon)

# Why it is a BAD idea




```
#include <iostream>
using namespace std;

int* f(int i) {
  static int array[3];
  // cout << "The static array lives at " << array << endl;
  array[0] = i;
  return array;
}

int main() {
  int * a;
  a = f(4); // we bind 'a' to the array built in f
  cout << a[0] << endl;

  int * b;
  b = f(666);
  cout << b[0] << endl;
  // .. alas, also "a" changed
  cout << a[0] << endl;
```